

Assignment 1:

Exploring OpenGL Programming

NAME:

STUDENT NUMBER:

EMAIL:

1 INTRODUCTION

This assignment details the implementation of key geometric and lighting components essential for a basic ray tracing renderer. The focus is on implementing robust ray intersection algorithms for primitives (triangle and AABB), accelerating scene traversal using a Bounding Volume Hierarchy (BVH), and employing the Monte Carlo integration method to accurately calculate direct lighting. The culmination of this work is the implementation of area light sources to generate the photorealistic visual effect of soft shadows.

2 IMPLEMENTATION DETAILS

2.1 Ray Intersection

(1) Triangle Intersection

To implement the ray-triangle intersection, we follow the steps that learned from the lecture slides. First, we start from the equation of the ray($o + td$) and the triangle(V_0, V_1, V_2).

$$o + td = (1 - u - v)V_0 + uV_1 + vV_2$$

Rearranging the equation, we have

$$(-d, V_1 - V_0, V_2 - V_0) \begin{pmatrix} t \\ u \\ v \end{pmatrix} = o - V_0$$

Using Cramer's rule, we can solve for t, u, v as follows:

$$t = \frac{(o - V_0) \cdot ((V_1 - V_0) \times (V_2 - V_0))}{d \cdot ((V_1 - V_0) \times (V_2 - V_0))}$$
$$u = \frac{d \cdot ((o - V_0) \times (V_2 - V_0))}{d \cdot ((V_1 - V_0) \times (V_2 - V_0))}$$
$$v = \frac{d \cdot ((V_1 - V_0) \times (o - V_0))}{d \cdot ((V_1 - V_0) \times (V_2 - V_0))}$$

After calculating t, u, v , we check if $t > 0, u > 0, v > 0, u+v < 1$ to determine if there is an intersection. The code implementation is as follows:

```
InternalVecType o = Cast<
    InternalScalarType>(ray.origin);
InternalVecType d = Cast<
    InternalScalarType>(ray.direction);

InternalVecType s = o - v0;
InternalVecType e1 = v1 - v0;
InternalVecType e2 = v2 - v0;
```

```
InternalVecType s1      = Cross(d, e2);
InternalVecType s2      = Cross(s, e1);
InternalScalarType denom = Dot(s1, e1);
if (abs(denom) < 1e-12) return false;
```

```
InternalScalarType t = Dot(s2, e2) /
    denom;
if (t < ray.t_min || t > ray.t_max)
    return false;
InternalScalarType u = Dot(s1, s) /
    denom;
if (u < 0) return false;
InternalScalarType v = Dot(s2, d) /
    denom;
if (v < 0 || u + v > 1) return false;
```

(2) AABB Intersection

To implement the ray-AABB intersection, we use the slab method. The idea is to find the intersection of the ray with the slabs defined by the AABB in each dimension. For each dimension, we calculate the entry and exit points of the ray with the slabs. The overall entry point is the maximum of the entry points in each dimension, and the overall exit point is the minimum of the exit points in each dimension. If the overall entry point is less than or equal to the overall exit point, there is an intersection. The code implementation is as follows:

```
Vec3f origin = ray.origin;
Vec3f inv_dir = ray.
    safe_inverse_direction;
Vec3f t0s = (low_bnd - origin) *
    inv_dir;
Vec3f t1s = (upper_bnd - origin) *
    inv_dir;
//Use ReduceMax and ReduceMin to find
//the overall tmin and tmax
Float tmin = ReduceMax(Min(t0s, t1s));
Float tmax = ReduceMin(Max(t0s, t1s));

if (tmin > tmax || tmax < 0) {
    return false;
}
```

```
*t_in = Max(0.0f, tmin);
```

1:2 • Name:
student number:
email:

```
*t_out = tmax;  
return true;
```

2.2 BVH Construction

In this homework, we only need to implement the part of stop criteria and median splitting. For the stop criteria, we check if the number of primitives in the current node is less than a threshold (e.g., 4). If it is, we stop splitting and make the current node a leaf node. For median splitting, in order to accelerate the construction process, we use function `nth_element()` to find the median of the centroids of the primitives along the chosen axis. This allows us to partition the primitives into two halves without fully sorting them, which improves efficiency. The code implementation is as follows:

```
// Stop criteria  
if (depth >= CUTOFF_DEPTH || span_right -  
    span_left <= 2) {  
    // create leaf node  
    const auto &node = nodes[span_left];  
    InternalNode result(span_left, span_right);  
    result.is_leaf = true;  
    result.aabb = prebuilt_aabb;  
    internal_nodes.push_back(result);  
    return internal_nodes.size() - 1;  
}  
// Median splitting  
std::nth_element(nodes.begin() + span_left,  
    nodes.begin() + split, nodes.begin() +  
    span_right, [dim](const NodeType &a, const  
        NodeType &b) {return a.getAABB().  
    getCenter()[dim] < b.getAABB().getCenter()  
    [dim];});
```

2.3 Refraction and Direct Lighting

For direct lighting, we implement the Monte Carlo integration method to estimate the direct lighting from area lights. We sample multiple points per pixel from the camera and then calculate the reflected light ray intersecting with the scene. For each intersection point, we performing shadow ray tests to determine if the light source is visible from the intersection point. If the light source is visible, we compute the contribution of the light source to the intersection point using a simple phong model. The code implementation is as follows:

```
// Monte Carlo direct lighting estimation  
const Vec2f &pixel_sample = sampler.  
    getPixelSample();  
auto ray = camera->generateDifferentialRay(  
    pixel_sample.x, pixel_sample.y);  
// Accumulate radiance  
assert(pixel_sample.x >= dx && pixel_sample.x  
    <= dx + 1);  
assert(pixel_sample.y >= dy && pixel_sample.y  
    <= dy + 1);
```

```
const Vec3f &L = Li(scene, ray, sampler);  
camera->getFilm()->commitSample(pixel_sample,  
    L);  
//...\dots  
// direct lighting  
// shadow ray test  
auto test_ray = interaction.spawnRay(  
    light_dir);  
test_ray.setTimeMax(dist_to_light);  
  
SurfaceInteraction shadow_it;  
if (scene->intersect(test_ray, shadow_it)) {  
    return color; //(0,0,0)  
}  
// compute contribution  
const Vec3f &light_position = light.position;  
const Vec3f &light_flux = light.flux;  
Float dist_to_light = Norm(light_position -  
    interaction.p);  
Vec3f light_dir = Normalize(  
    light_position - interaction.p);  
auto test_ray = DifferentialRay(interaction.p,  
    light_dir);  
test_ray.setTimeMax(dist_to_light);  
SurfaceInteraction shadow_it;  
if (scene->intersect(test_ray, shadow_it)) {  
    continue;  
}  
Float cos_theta = std::max(Dot(light_dir,  
    interaction.normal), 0.0f); // one-sided  
Vec3f albedo = bsdf->evaluate(interaction);  
Vec3f contribution = albedo * cos_theta * (  
    light_flux / (dist_to_light *  
    dist_to_light));  
color += contribution;
```

For the refraction part, we implement the recursive ray tracing method to handle reflective surfaces. When a ray intersects with a reflective surface, we compute the reflection direction based on the surface normal and the incoming ray direction. We then spawn a new ray in the reflection direction and trace the ray through transparent objects until it first intersects a non-transparent (solid) object. The code implementation is as follows:

```
// in integrator.cpp:  
Vec3f IntersectionTestIntegrator::Li(ref<  
    Scene> scene, DifferentialRay &ray,  
    Sampler &sampler) const {  
    Vec3f color(0.0);  
    //...  
    if (is_perfect_refraction) {  
        interaction.bsdf->sample(interaction,  
            sampler, nullptr);
```

```

        ray = interaction.spawnRay(interaction.wi
        );
        continue;
    }
    //...
    color = directLighting(scene, interaction,
        sampler);
    return color;
}
// in bsdf.cpp:
Vec3f PerfectRefraction::sample(
    SurfaceInteraction &interaction, Sampler
    &sampler, Float *pdf) const {
    // The interface normal
    Vec3f normal = interaction.shading.n;
    // Cosine of the incident angle
    Float cos_theta_i = Dot(normal, interaction
        .wo);
    // Whether the ray is entering the medium
    bool entering = cos_theta_i > 0;
    // Corrected eta by direction
    Float eta_corrected = entering ? eta : 1.0F
        / eta;

    Vec3f wi = Vec3f(0.0);
    bool refracted = Refract(interaction.wo,
        normal, eta_corrected, wi);
    if (!refracted) {
        // Total internal reflection
        interaction.wi = Reflect(interaction.wo,
            normal);
    } else {
        interaction.wi = wi;
    }
    // Set the pdf and return value, we dont
    // need to understand the value now
    if (pdf != nullptr) *pdf = 1.0F;
    return Vec3f(1.0);
}

```

The final performance is shown in Figure 1.

2.4 Multiple Light Sources

To handle multiple light sources, first we need to modify the scene representation to store a list of light sources. Then, during the rendering process, we iterate over each light source and compute its contribution to the final color at each intersection point. The code implementation is as follows:

```

// in integrator.h:
if (props.hasProperty("point_lights")) {
    const auto &lights_props =

```

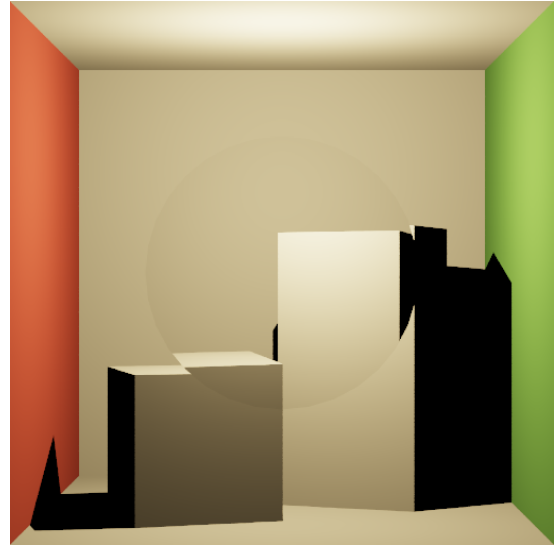


Fig. 1. Direct Lighting With Single PointLight Source

```

        props.getProperty<std::vector<Properties
        >>("point_lights");
        for (const auto &light_prop : lights_props)
        {
            PointLight light;
            light.position = light_prop.getProperty<
                Vec3f>("position");
            light.flux = light_prop.getProperty<
                Vec3f>("flux");
            point_lights.push_back(light);
        }
    }
    // read the multiple point lights from json
    file
    else {
        PointLight light;
        light.position = props.getProperty<Vec3f>("
            point_light_position", Vec3f(0.0F, 5.0F
            , 0.0F));
        light.flux = props.getProperty<Vec3f>("
            point_light_flux", Vec3f(1.0F, 1.0F,
            1.0F));
        point_lights.push_back(light);
    }
}
// in integrator.cpp:
if (!point_lights.empty()) {
    for (const auto &light : point_lights) {
        // compute contribution from each point
        light
    }
}

```

```

1:4 • Name:
student number:
email:
// in json file:
"point_lights": [
{"position": [ 0.0, 1.5, 0.5], "flux": [
17.0, 12.0, 5.0]},
{"position": [ 0.0, 1.0, 1.0], "flux": [
17.0, 10.0, 5.0]}]

```

I implemented two point light sources in the scene to demonstrate the effect of multiple light sources. The final performance is shown in Figure 2. We can clearly see the multiple shadows casted by the two light sources.

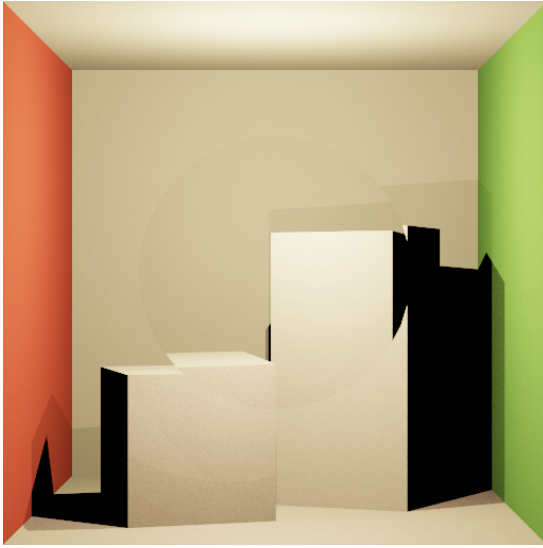


Fig. 2. Direct Lighting With Multiple PointLight Sources

2.5 Area Light with Soft Shadow

To implement area lights with soft shadows, I found the project skeleton has already provided the area light class. So I mainly focused on implementing the Monte Carlo integration method to estimate the direct lighting from area lights. I sampled multiple points on the area light source and calculated the reflected light ray intersecting with the scene. For each intersection point, I performed shadow ray tests to determine if the light source is visible from the intersection point. If the light source is visible, I computed the contribution of the light source to the intersection point using the rendering equation. The contribution calculation is as follows:

$$L_{dir} = \int_A f_r(p, \omega_i, \omega_o) L_e(p', -\omega_i) \cos \theta_i \frac{|\cos \theta_o|}{|p - p'|^2} dA'$$

Where L_{dir} is the direct lighting at point p , f_r is the BRDF at point p , L_e is the emitted radiance from the area light at point p' , θ_i is the angle between the incoming light direction and the surface normal at point p , θ_o is the angle between the outgoing light direction and the surface normal at point p' , and A is the area of the light source. The code implementation is as follows:

```

Vec3f IntersectionTestIntegrator::
    directLighting(ref<Scene> scene,
        SurfaceInteraction &interaction, Sampler
        &sampler) const {
    Vec3f color(0, 0, 0);
    const BSDF *bsdf = interaction.bsdf;
    bool is_ideal_diffuse = dynamic_cast<const
        IdealDiffusion *>(bsdf) != nullptr;
    if (bsdf != nullptr && is_ideal_diffuse) {
        const auto &area_lights = scene
            ->getLights();
        constexpr int AREA_LIGHT_SAMPLES = 8;
        if (!area_lights.empty()) {
            for (const auto &light : area_lights) {
                const AreaLight *area_light =
                    dynamic_cast<const AreaLight *>(
                        light.get());
                if (!area_light) {
                    continue;
                }
                Vec3f L_dir(0.0);

                for (int i = 0; i <
                    AREA_LIGHT_SAMPLES; ++i) {
                    SurfaceInteraction
                        light_interaction = area_light
                            ->sample(interaction, sampler);

                    Vec3f light_dir = Normalize(
                        light_interaction.p -
                        interaction.p);
                    Float dist = Norm(
                        light_interaction.p -
                        interaction.p);

                    auto test_ray = interaction.
                        spawnRayTo(light_interaction.p);
                    ;
                    SurfaceInteraction shadow_it;
                    if (scene->intersect(test_ray,
                        shadow_it)) {
                        continue;
                    }

                    Float cos_theta_i = Dot(light_dir,
                        interaction.normal); // $cos\
                    theta_i$
                    Float cos_theta_o = Dot(-light_dir,
                        light_interaction.normal); //
                    $cos\theta_o$

```

```

if (cos_theta_i <= 0 || cos_theta_o
    <= 0) {
    continue;
}
Vec3f f_r = bsdf->evaluate(
    interaction);
// G = |cos_theta_o| / (|p - p'|^2)
Float G = cos_theta_o / (dist *
    dist);
Vec3f Le = area_light->Le(
    light_interaction, -light_dir);
Vec3f contribution = f_r * Le *
    cos_theta_i * G * (1.0F /
    light_interaction.pdf);

L_dir += contribution;
}

color += L_dir / AREA_LIGHT_SAMPLES;
}
}
}
return color;
}

```

And the final performance is shown in Figure 3. We can see the soft shadows casted by the area light source.

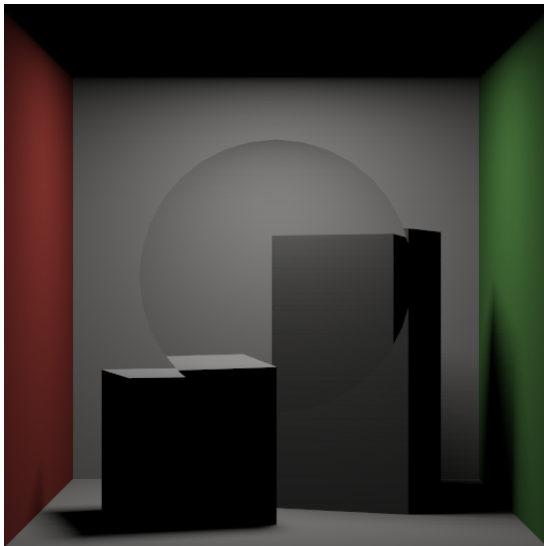


Fig. 3. Area Light with Soft Shadows

3 RESULTS

The implemented ray tracing core components including ray-triangle and AABB intersections, and the BVH acceleration structures successfully processed the test scene geometry, ensuring accurate light-geometry interactions.

As shown in Figure 1 and Figure 2, the direct lighting estimation using single and multiple point light sources correctly generates sharp, distinct hard shadows, validating the implementation of the shadow ray tests and basic diffuse contribution calculation.

The final integration of the area light sampling method, detailed in Section 2.5 and demonstrated in Figure 3, successfully estimates the direct illumination using Monte Carlo area sampling. This approach yields the intended visual effect of soft shadows, where shadows smoothly transition from dark umbra to light penumbra, thereby validating the use of the $\frac{f_r L_e |\cos \theta_i| G}{pdf_A}$ formula for direct lighting estimation